

System Requirements

Template

Version 2.1 - April 02, 2026

Using This Template

This and other PDM tools are available. All Sections are required to be addressed, however if a section or subsection is not needed, that section/subsection of the document can be marked as Not Applicable but as explanation must be provided as to why it does not apply. Please also reference the **Lessons Learned** section in the Appendix for additional information that may assist.

To create a deliverable from this template:

1. Delete the template title page (previous page) and this page.
2. Replace [bracketed text] on the cover page (next page) with your project and agency information.
3. Replace [bracketed text] in the tool header area at the top of Page i (Contents page) with the same project and agency information as on the cover page.

Note: Please do not remove or modify content in the footer area.

4. Complete the entire template. Each section contains abbreviated instructions, shown in italics, and a content area. The content area is marked with a placeholder symbol ($\Rightarrow$) or with a table. Relevant text from other project deliverable may be pasted into content areas.

Note: Please do not remove the italicized instructions.

5. Update the table of contents by right-clicking and selecting "Update Field," then "Update entire table."

Template Revision History

Version	Date	Name	Description
1.0	2/5/2012	PDM Project team	Initial Creation
2.0	6/28/2012	Elizabeth Bradley/David Davis	Reformatted document template; Renamed template to System Requirements. Added Initial Technical Architecture Specifications
2.1	1/14/2013	David Davis	Revised system requirements instructions section
2.1	5/17/2013	David Davis	Correct typos

Project Delivery Methodology (PDM)

System Requirements

[Functional Office(s) Name]

[Btech AIDA]

VERSION: 1.0

REVISION DATE:

Approval of the System Requirements indicates an understanding of the purpose and content described in this deliverable. By signing this deliverable, each individual agrees with the content contained in this deliverable.

Approver Name	Title	Signature	Date
S M Nitish	TrainHub		

Contents

Section 1 Purpose.....	3
Section 2 General System Requirements.....	3
2.1 Major System Capabilities.....	Error! Bookmark not defined.
2.2 Major System Conditions.....	Error! Bookmark not defined.
2.3 System Interfaces.....	Error! Bookmark not defined.
2.4 System User Characteristics.....	Error! Bookmark not defined.
Section 3 Policy and Regulation Requirements	5
3.1 Policy Requirements	6
3.2 Regulation Requirements	6
Section 4 Security Requirements	Error! Bookmark not defined.
Section 5 Training Requirements	7
Section 6 Initial Capacity Requirements	Error! Bookmark not defined.
Section 7 Initial System Architecture	Error! Bookmark not defined.
Section 8 System Acceptance Criteria	Error! Bookmark not defined.
Section 9 Current System Analysis.....	12
Section 10 References.....	Error! Bookmark not defined.
Section 11 Glossary	Error! Bookmark not defined.
Section 12 Document Revision History	Error! Bookmark not defined.
Section 13 Appendices	16

Section 1 Purpose

The purpose of this System Requirements Document is to clearly define and document the complete set of functional and non-functional requirements that will guide the design, development, testing, deployment, and maintenance of the TrainHub application. TrainHub is a web-based corporate training management platform developed using the MERN stack (MongoDB, Express.js, React, and Node.js). The primary objective of this system is to replace traditional manual training management methods such as spreadsheets, emails, and disconnected records with a centralized, automated, and scalable digital platform.

This document establishes the major system capabilities required for TrainHub, including user registration and authentication, course creation and publishing, employee self-enrollment, progress tracking, dashboard reporting, and role-based access for different categories of users such as Administrators, Trainers, and Employees. It also defines the system architecture, describing the interaction between the React frontend, Node.js/Express backend services, and MongoDB database to ensure seamless communication, efficient data storage, and responsive user experience.

In addition, the document specifies essential security requirements to protect sensitive organizational and user data. These include secure login mechanisms using JSON Web Tokens (JWT), password encryption using hashing methods, Role-Based Access Control (RBAC), input validation, and protection against unauthorized access. Initial capacity requirements are also identified to ensure the system can support multiple concurrent users, increasing training records, and future scalability as the organization expands.

Furthermore, this document defines the system acceptance criteria agreed upon by HR administrators, trainers, management, and other key stakeholders. These criteria ensure that the final TrainHub application successfully meets business objectives such as automating employee training enrollments, improving onboarding efficiency, enabling accurate compliance tracking, reducing administrative workload, and providing real-time visibility into training performance metrics. Overall, this document serves as the foundation for delivering a reliable, secure, and efficient training management solution.

Section 2: General System Requirements

2.1 Major System Capabilities

- **Web Accessibility:** The system must be accessible via modern web browsers (Chrome, Firefox, Edge, Safari) without requiring client-side installation.
- **Responsive Design:** The interface must be fully responsive, ensuring optimal viewing and interaction across desktop monitors, tablets, and mobile devices for "learning on the go."
- **High Availability:** The system should target high uptime (e.g., 24/7 availability) to accommodate employees training across different corporate shifts.
- **Role-Based Security:** The system must automatically enforce Role-Based Access Control (RBAC) using JSON Web Tokens (JWT) to ensure users only access their authorized dashboards (Admin, Trainer, or Employee).
- **Real-Time Tracking:** The system must provide immediate, real-time updates to enrollment numbers and completion statistics on the Admin dashboard.

2.2 Major System Conditions

- **Technology Stack Constraint:** The application must strictly utilize the MERN stack (MongoDB, Express.js, React.js, Node.js).
- **Authentication Dependency:** The system assumes the client browser allows local storage functionality, as it is required to store the JWT for maintaining active user sessions.
- **Deployment Environment:** The backend must be compatible with standard Node.js hosting environments (e.g., Render, Heroku, AWS), and the database must utilize a NoSQL document structure (e.g., MongoDB Atlas).

2.3 System Interfaces

- **TrainHub React Frontend (Client Interface):** The client-side application communicates exclusively with the backend via RESTful API calls sending and receiving JSON payloads.
- **TrainHub Express Backend (Server Interface):** The Node.js server acts as the intermediary, securely processing API requests and enforcing business logic.
- **MongoDB Atlas (Database Interface):** The backend interfaces with an external MongoDB cluster via the Mongoose ODM library to securely execute CRUD (Create, Read, Update, Delete) operations for Users, Courses, and Enrollments.

2.4 System User Characteristics

User Type	Function	Location/Device	Est. Quantity	Nature of Use
Administrators (HR / Management)	Monitor system health, view global enrollment analytics, manage overall user access.	Office / Desktop	2 - 5 users	Daily monitoring and reporting; requires high technical proficiency.
Instructors (Trainers)	Create, edit, manage, and publish training modules.	Office / Desktop	10 - 20 users	Frequent interaction to update course catalogs; moderate technical proficiency.
Employees (Learners)	Browse available courses, self-enroll, and track personal training progress.	Office & Remote / Desktop & Mobile	100 - 500+ users	Intermittent use during training cycles; diverse technical proficiency requiring an intuitive, simple UI.

Type equation here.

Section 3 Policy and Regulation Requirements

Specify relevant applicable laws, regulations, policies, and standards that will affect the operation and performance of the system, as well as any relevant external regulatory requirements, or constraints imposed by normal business practices.

3.1 Policy Requirements

⇒

3.2 Regulation Requirements

⇒

Section 4: Security Requirements

4.1 User Authentication

- **Secure Credentials:** Users must authenticate using a registered email address and password.
- **Password Hashing:** Passwords must be securely hashed and salted using cryptographic algorithms (e.g., bcrypt) prior to database insertion. The system will never store plain-text passwords.
- **Token-Based Authentication:** Upon successful login, the system will generate a JSON Web Token (JWT). This token will be used to securely identify the user during subsequent requests without needing to repeatedly query the database for credentials.

4.2 Authorization & Role-Based Access Control (RBAC)

- **Strict Role Enforcement:** The system must enforce Role-Based Access Control at both the frontend (UI routing) and backend (API endpoint) levels. The designated roles are: Administrator, Instructor, and Employee.
- **Protected Endpoints:** Backend API routes must utilize custom security middleware to decode the user's JWT and verify their assigned role before granting access to sensitive operations (e.g., ensuring only an "Instructor" role can hit the "Create Course" endpoint).

4.3 Data Protection & Integrity

- **Data in Transit:** All client-server data transmission should occur over secure HTTPS/TLS protocols in the live production environment to prevent interception (Man-in-the-Middle attacks).
- **Input Sanitization:** The backend must perform strict input validation using Mongoose schema definitions to reject malicious payloads and prevent NoSQL Injection or Cross-Site Scripting (XSS) vulnerabilities.

4.4 Session Management

- **Token Expiration:** JWTs must be issued with a defined expiration limit to automatically log out idle users and reduce the window of opportunity for session hijacking.
- **Secure Client Storage:** The client-side application (React) must manage the JWT securely and discard it entirely upon the user triggering the "Log Out" function, terminating the session.

Section 5: Training Requirements

The TrainHub system is designed with modern, intuitive UI/UX principles to minimize the learning curve. However, targeted training is required based on user roles to ensure smooth adoption across the organization:

5.1 Administrator Training (HR / IT Staff)

- **Requirement:** Administrators require technical onboarding to understand how to provision initial user accounts, assign roles, and interpret the global analytics dashboard.

- **Format:** A 1-hour live walkthrough session and a technical handover document detailing the backend environment and MongoDB database structure.

5.2 Instructor Training (Trainers)

- **Requirement:** Instructors must understand how to access the Trainer Dashboard, create new course modules, define course capacities, and publish training materials to the employee portal.
- **Format:** A short (10-15 minute) pre-recorded video tutorial and a 1-page quick reference guide (PDF) distributed via email prior to system launch.

5.3 Employee Training (Learners)

- **Requirement:** The Employee portal mimics standard, modern web applications (e.g., e-commerce or streaming platforms), meaning no formal, synchronous training is required. Employees only need to know how to log in, browse the catalog, and click "Enroll."
- **Format:** A simple "Welcome to TrainHub" introductory email containing their login credentials and a 3-step visual infographic showing how to find and enroll in their first course.

Type equation here.

Section 6: Initial Capacity Requirements

The initial capacity requirements for the TrainHub system are designed to support a mid-sized corporate environment. Estimations are based on a projected user base of approximately 500 Employees, 20 Instructors, and 5 System Administrators.

6.1 Transaction Estimations & Processing Frequency

- **Lowest Expected Usage (Standard Daily Operations):** During standard mid-quarter operations, the system expects low-frequency traffic. Estimated at 10 to 30 API transactions (logins, course browsing, and enrollments) per day.
- **Highest Expected Usage (Seasonal Peaks):** The system anticipates significant traffic spikes at the end of financial quarters and during annual

compliance training deadlines. During these peak periods, the system must process up to 300-400 concurrent user sessions and up to 1,500 API transactions per hour as employees rapidly complete mandatory training modules.

6.2 Storage & Memory Requirements

- **Database Storage (MongoDB):** Because the system relies on NoSQL documents (storing primarily text-based metadata for users, course descriptions, and enrollment records rather than hosting heavy raw video files), data footprint is extremely lightweight. An initial database storage capacity of 500MB to 1GB is more than sufficient for the first 1-2 years of operation.
- **Server Memory (RAM):** The Node.js/Express backend environment is highly efficient. To comfortably handle the estimated seasonal peaks without bottlenecking, the application server requires an initial allocation of 1GB to 2GB of RAM.

Type equation here.

Section 7: Initial System Architecture:

7.1 Data Platform & Environment

The TrainHub application is built on a distributed, cloud-ready architecture utilizing the MERN stack. The backend server environment runs on Node.js, and the data platform relies on a NoSQL document database (MongoDB) to handle dynamic corporate training data flexibly and efficiently.

A. Specialized Hardware Requirements

- No specialized hardware is required to be purchased or upgraded.
- **End-Users:** Only require standard commercial hardware (desktop computers, laptops, or mobile devices) with a stable internet connection.
- **Server Implementation:** The system will be deployed to scalable, virtualized cloud infrastructure (e.g., AWS, Render, or Heroku), negating the need for on-premise physical servers.

B. Specialized Software Requirements

- **End-Users:** No specialized software installations are required. Users access the system entirely via standard, modern web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari).
- **Development/Hosting:** A cloud-hosted MongoDB cluster (e.g., MongoDB Atlas) must be provisioned prior to deployment to handle database operations without requiring local software installation on the host server.

C. Programming Languages and Tools:

The development of the application utilizes a unified, full-stack JavaScript approach:

- **Core Languages:** JavaScript (ES6+), HTML5, and CSS3.
- **Frontend Framework:** React.js (bootstrapped and bundled via Vite for rapid development).
- **Backend Framework:** Node.js paired with Express.js for RESTful API construction.
- **Database Management:** MongoDB, utilizing the Mongoose Object Data Modeling (ODM) library.
- **Development Tools:** Git (version control), VS Code (IDE), Postman (API endpoint testing), and ESLint (code quality and formatting).

D. Network and Operating System Requirements

- **Development Environment:** The application is OS-agnostic for development and can be built on Windows, macOS, or Linux operating systems.
- **Production Environment:** The live application is targeted for deployment on a standard Linux-based cloud server (e.g., Ubuntu) running the Node.js runtime environment over standard TCP/IP networks via HTTPS (Port 443) for secure communication.

Type equation here.

Section 8: System Acceptance Criteria

The final end product will be formally accepted by the project sponsor and key HR/Management stakeholders upon the successful demonstration and fulfillment of the following criteria:

- **Parallel Operation Phase:** The new TrainHub system must run in parallel with the current legacy tracking methods (e.g., manual spreadsheets/emails) for a period of **1 month** (or one complete training cycle) to verify enrollment accuracy and system stability without disrupting ongoing operations.
- **Historical Data Migration:** A minimum of **2 years** of historical employee training records must be successfully converted and populated into the new MongoDB database on day one of the parallel run.
- **Role-Based Security Verification:** Independent testing must confirm that Role-Based Access Control (RBAC) is strictly enforced, specifically proving that standard Employee accounts cannot access the Trainer or Administrator dashboards under any circumstances.
- **Core Functional Completeness:** The system must successfully demonstrate the complete "training lifecycle" without errors: an Instructor creates a course, an Employee successfully enrolls via the UI, and the Admin dashboard instantly updates to reflect the new statistical data.
- **Performance Benchmark:** The application must maintain an average API response time of under 1 second during standard daily operations to ensure a seamless User Experience (UX).

Section 9: Current System Analysis

9.1 Description of Current System

The organization currently does not utilize a centralized Learning Management System (LMS). The "current system" consists of highly decentralized, manual processes relying primarily on static spreadsheets (e.g., Microsoft Excel) and email chains to manage corporate training.

9.2 How the Current System is Used

- **Course Announcements:** Instructors inform HR of a new training session. HR then manually sends a company-wide email detailing the course.

- **Enrollment Flow:** Employees must reply to the email or fill out a separate online form to express interest.
- **Record Keeping:** An HR administrator acts as the central bottleneck, manually reading emails and copying employee data into a master "Training Tracker" spreadsheet. Instructors must then request a copy of this spreadsheet to see who is attending their class.

9.3 System Interfaces & Data Flows

- **Interfaces:** The current system operates in complete isolation with zero automated interfaces to other enterprise software.
- **Data Flow:** Data flows linearly and manually: Instructor -> HR (Email) -> Employee (Email) -> HR (Email) -> Master Spreadsheet. This results in frequent data entry errors and lost enrollment requests.

9.4 Reporting Capability

Current reporting capabilities are extremely limited and labor-intensive. To generate a quarterly compliance report or view overall completion statistics, an administrator must manually aggregate data across multiple spreadsheet tabs, utilize complex VLOOKUPs, and build pivot tables. This process often takes hours or days to compile, meaning management rarely has access to real-time training analytics.

9.5 Data Conversion Requirements

To transition to the new TrainHub system, the legacy "Master Training Tracker" spreadsheet must undergo data cleansing. The unstructured spreadsheet data must be formatted and mapped to match the new MongoDB NoSQL schemas (specifically the Users and Enrollments collections). This cleansed data will then be converted to JSON or CSV format and batch-imported into the new database prior to system launch to ensure historical tracking continuity.

Section 10: References

The following documents, standards, and sources of information were utilized during the development of this System Requirements Specification:

Document No.	Document Title	Date	Author / Responsible Office
ISO-29148	ISO/IEC/IEEE 29148:2018 (Requirements Engineering Life Cycle Processes)	2018	Int. Organization for Standardization
TH-DOC-01	TrainHub Initial Project Proposal & Charter	Oct 2023	Project Management Team
TH-DOC-02	Legacy Training Tracker Process Analysis	Nov 2023	HR Administration Dept.
REF-MERN-01	React.js Official Documentation (v18.x)	2023	Meta / React Open Source Team
REF-MERN-02	Express.js API Reference & Security Best Practices	2023	OpenJS Foundation

Section 11: Glossary

- **API (Application Programming Interface):** A set of rules and protocols that allows the React frontend to communicate with and request data from the Node.js backend.
- **DAST (Dynamic Application Security Testing):** A process of testing an application from the outside in while it is running to find security vulnerabilities (like XSS or broken authentication).
- **JWT (JSON Web Token):** A secure, compact, and self-contained method for securely transmitting information between parties as a JSON object. Used in this system for user authentication and session management.

- **MERN Stack:** The software stack used to build the TrainHub application, consisting of MongoDB (database), Express.js (backend framework), React.js (frontend library), and Node.js (runtime environment).
- **NoSQL:** A type of database (like MongoDB) that provides a mechanism for storage and retrieval of data that is modeled in means other than the tabular relations used in traditional SQL relational databases.
- **RBAC (Role-Based Access Control):** A method of restricting network or system access based on the roles of individual users within an enterprise (e.g., Administrator, Instructor, Employee).
- **RESTful (Representational State Transfer):** An architectural style for designing networked applications. It relies on a stateless, client-server protocol, usually HTTP, treating server data as resources that can be created, read, updated, or deleted.
- **SPA (Single Page Application):** A web application or website that interacts with the user by dynamically rewriting the current web page with new data from the web server, instead of the default method of a web browser loading entire new pages.
- **SRS (System Requirements Specification):** This document itself; a comprehensive description of the intended purpose and environment for software under development.

⇒

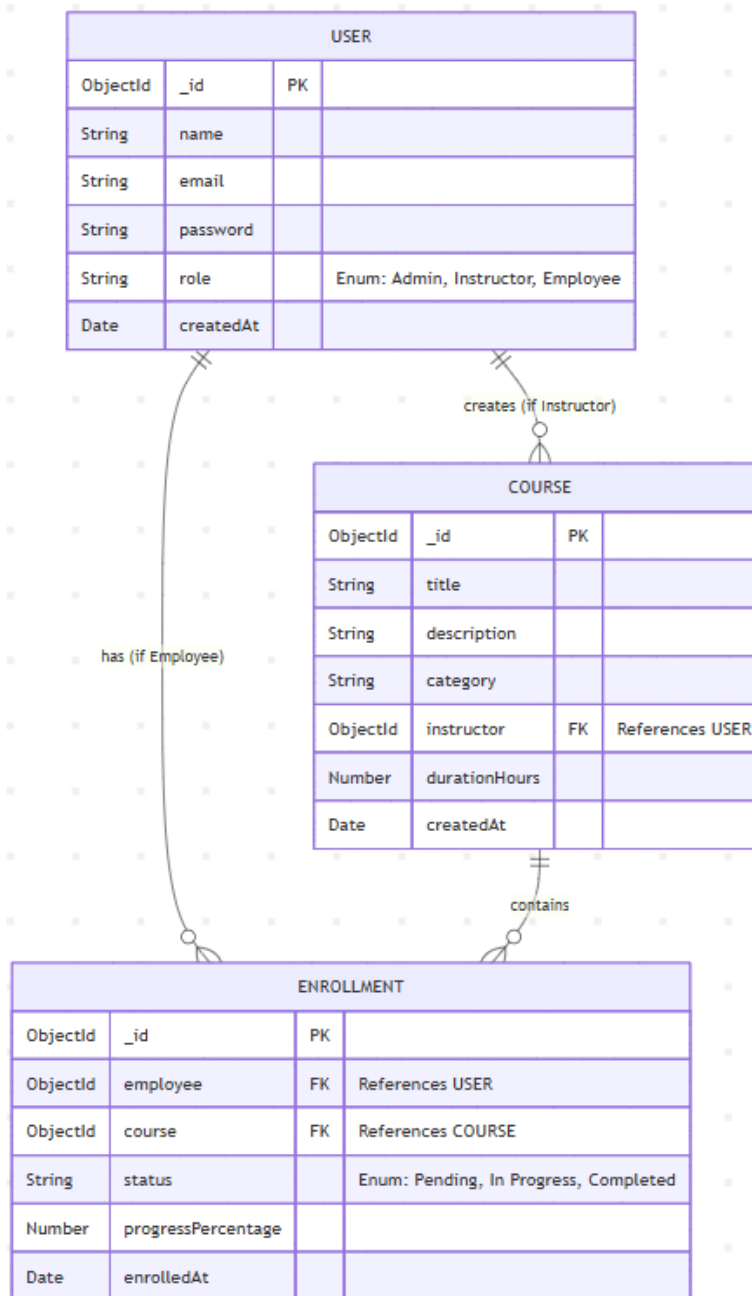
Section 12: Document Revision History

Version	Date	Name	Description
1.0	Oct 10, 2023	Project Team	Initial creation and drafting of core functional requirements and system architecture for TrainHub.
1.1	Oct 18, 2023	Project Team	Updated Section 4 (Security Requirements) to include specific JWT and RBAC enforcement details following IT security review.
1.2	Nov 02, 2023	Project Team	Finalized Section 8 (Acceptance Criteria) and Section 6 (Capacity Requirements) based on feedback from HR Management stakeholders.
2.0	Nov 15, 2023	Project Team	Final approved version authorized by Project Sponsor for commencement of development phase.

Section 13 Appendices

The following appendices contain supplementary diagrams, technical specifications, and visual references that support the requirements outlined in this document.

Appendix A: Entity-Relationship Diagram (ERD)



Appendix B: User Case and Sequence:

